



第三章 数据表示

秦磊华 计算机学院

3.1 常见编码

问题提出：

- 如何快速实现二进制数与十进制数之间的转换？
- 为什么从硬件层面考虑提高系统的可靠性？有哪些提高可靠性的方法？



3.1 常见编码

1. BCD编码

- 概念

用四位二进制数表示一位十进制数的编码(BCD: Binary Coded Decimal);

- BCD码既有二进制的形式，又有十进制的特点;
- 常见类型: 8421码、2421码和余3码。



3.1 常见编码

1. BCD编码– 8421码

- 用4位二进制码表示一位十进制字符的一种有权码，4位二进制码从高位至低位的权依次为8、4、2、1。
- 8421码编码的0 ~ 9与用4位二进制数表示的0 ~ 9相同,减少了二进制数与十进制数之间转换的环节,故是一种人机联系时广泛使用的中间形式。
- 不允许出现1010 ~ 1111六种组合(因为没有十进制数字符号与其对应)。
- 十进制数字字符0 ~ 9的8421码与相应ASCII码的低四位相同，有利于简化输入输出过程中BCD码与字符代码的转换。



3.1 常见编码

ASCII 字符代码表 一

高四位 低四位		ASCII非打印控制字符										ASCII 打印字符											
		0000					0001					0010		0011		0100		0101		0110		0111	
		0					1					2		3		4		5		6		7	
		+进制	字符	ctrl	代码	字符解释	+进制	字符	ctrl	代码	字符解释	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符
0000	0	0	BLANK NULL	^@	NUL 空	16	▶	^P	DLE 数据链路转意	32		48	0	64	@	80	P	96	`	112	p		
0001	1	1	☺	^A	SOH 头标开始	17	◀	^Q	DC1 设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q		
0010	2	2	☹	^B	STX 正文开始	18	↕	^R	DC2 设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r		
0011	3	3	♥	^C	ETX 正文结束	19	!!	^S	DC3 设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s		
0100	4	4	♦	^D	EOT 传输结束	20	¶	^T	DC4 设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t		
0101	5	5	♣	^E	ENQ 查询	21	♫	^U	NAK 反确认	37	%	53	5	69	E	85	U	101	e	117	u		
0110	6	6	♠	^F	ACK 确认	22	■	^V	SYN 同步空闲	38	&	54	6	70	F	86	V	102	f	118	v		
0111	7	7	●	^G	BEL 震铃	23	↕	^W	ETB 传输块结束	39	'	55	7	71	G	87	w	103	g	119	w		
1000	8	8	◼	^H	BS 退格	24	↑	^X	CAN 取消	40	(	56	8	72	H	88	X	104	h	120	x		
1001	9	9	○	^I	TAB 水平制表符	25	↓	^Y	EM 媒体结束	41	)	57	9	73	I	89	Y	105	i	121	y		
1010	A	10	◻	^J	LF 换行/新行	26	→	^Z	SUB 替换	42	*	58	:	74	J	90	Z	106	j	122	z		
1011	B	11	♂	^K	VT 垂直制表符	27	←	^[	ESC 转意	43	+	59	;	75	K	91	[	107	k	123	{		
1100	C	12	♀	^L	FF 换页/新页	28	└	^\	FS 文件分隔符	44	,	60	<	76	L	92	\	108	l	124			
1101	D	13	🎵	^M	CR 回车	29	↔	^]	GS 组分隔符	45	-	61	=	77	M	93	]	109	m	125	}		
1110	E	14	🎵	^N	SO 移出	30	▲	^6	RS 记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~		
1111	F	15	◼	^O	SI 移入	31	▼	^-	US 单元分隔符	47	/	63	?	79	O	95	_	111	o	127	Δ	Back Space	

注：表中的ASCII字符可以用:ALT + “小键盘上的数字键”输入

3.1 常见编码

1. BCD编码– 8421码

例1 8421码与十进制数之间的互换

- 按位进行，即十进制数的每一位与4位二进制编码对应。

$$(258)_{10} = (0010\ 0101\ 1000)_{8421\text{码}}$$

$$(258)_{10} = (100000010)_2 = (0010\ 0101\ 1000)_{8421\text{码}}$$

新问题: 两个数的BCD 码能直接实现加法运算? (16+18)

- BCD加法: 节省了十一二转换时间, 有利于提高运算器速度

BCD码加(减)运算修正的条件和方法是:

- ①低4位向高4位进(借)位, 低4位加(减)6修正;
- ②低4位出现非法码, 低4位加(减)6修正;
- ③高4位出现非法码, 高4位加(减)6修正;
- ④高4位向更高位进(借)位, 高4位加(减)6修正;



3.1 常见编码

1. BCD编码– 2421码

- 用4位二进制表示一位十进制字符的一种有权码，4位二进制码从高位至低位的权依次为2、4、2、1。
- 一个十进制数字字符X的2421码为 $a_4a_3a_2a_1$, 则对应的十进制的值为：
$$X = 2a_4 + 4a_3 + 2a_2 + a_1$$
- 不具备单值性：0101、1011 都对应十进制数 5。
- 为保持与十进制数的唯一对应，2421码不使用0101 ~ 1010等6种状态？
- 是一种对9的自补代码：2421码按位取反, 便可得到该数对9补数的2421码



3.1 常见编码

1. BCD编码– 2421码

不出现中间的6种状态

0101
0110
0111
1000
1001
1010

十进制字符	2421码
0	0000
1	0001
2	0010
3	0011
4	0100
5	1011
6	1100
7	1101
8	1110
9	1111

- 两个2421码相加时，若出现最高位进位的伪码组，则减0110调整（16+16）
- 两个2421码相加时，若出现伪码组但未出现最高位进位，则加0110调整



3.1 常见编码

1. BCD编码- 余3码

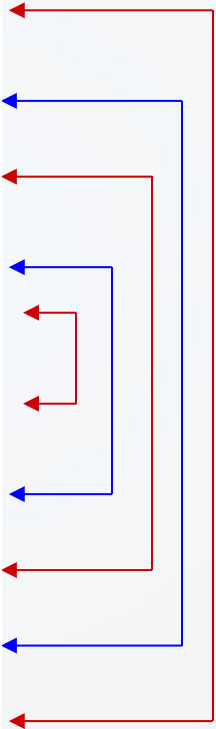
- 在8421码基础上每位十进制数BCD码再加上二进制数0011得到;
- 是一种有权码还是无权码!
- 余3码中无0000 ~ 0010、1101 ~ 1111这6种代码;
- 余3码具有自补性, 余3码也是一种“对9的自补码”



3.1 常见编码

1. BCD编码- 余3码

十进制字符	余3码
0-0000	0011
1-0001	0100
2-0010	0101
3-0011	0110
4-0100	0111
5-0101	1000
6-0110	1001
7-0111	1010
8-1000	1011
9-1001	1100



3.2 可靠性编码

问题提出:

- 为什么要考虑可靠性编码问题?
- 可靠性编码如何编码?如何体现可靠性?有何特点?



3.2 可靠性编码

1.引入可靠性编码的原因分析

- 自然界存在很多干扰;
- 二进制数据在被处理、传输、存储的过程中受干扰易发生错误;
- 其他原因!

2.为什么要在编码层引入可靠性检测?

- 数据是被处理、传输和存储的直接对象;
- 当直接对象出现错误时若不能检测, 高层检测将会付出更大的代价! (将问题解决在基层)
- 编码与数据在同一层,可以提供参考信息,便于检错与纠错。



3.2 可靠性编码

3.格雷(Gray)码

- 从二进制数到格雷码编码规则

设二进制码为 $B = B_{n-1}B_{n-2} \cdots B_{i+1}B_i \cdots B_1B_0$, 对应格雷码为 $G = G_{n-1}G_{n-2} \cdots G_{i+1}G_i \cdots G_1G_0$

$$G_{n-1} = B_{n-1}$$

$$G_i = B_{i+1} \oplus B_i \quad 0 \leq i \leq n-2$$

其中, 运算“ $\oplus$ ”称为“异或”运算, 运算规则是:

$$0 \oplus 0 = 0; \quad 0 \oplus 1 = 1;$$

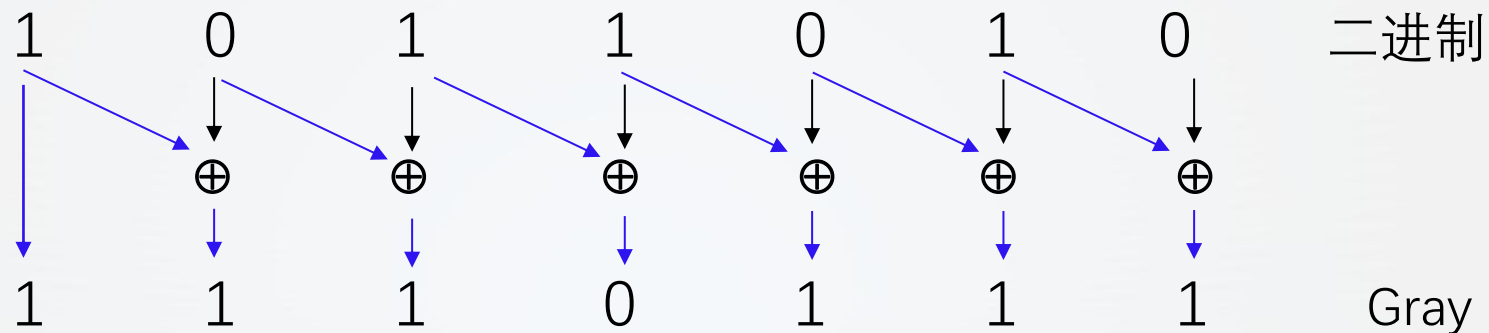
$$1 \oplus 0 = 1; \quad 1 \oplus 1 = 0。$$

3.2 可靠性编码

3.格雷(Gray)码

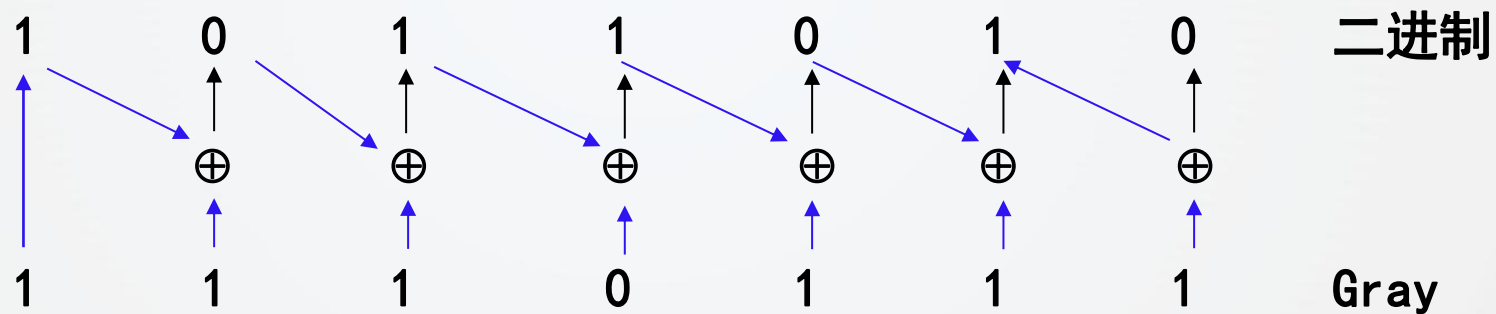
$$G_{n-1} = B_{n-1}$$

$$G_i = B_{i+1} \oplus B_i$$



$$B_{n-1} = G_{n-1}$$

$$B_i = B_{i+1} \oplus G_i$$

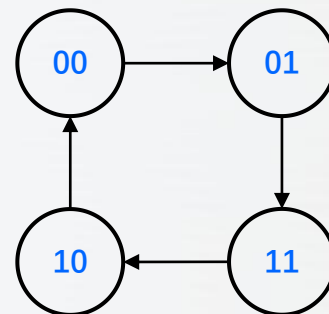
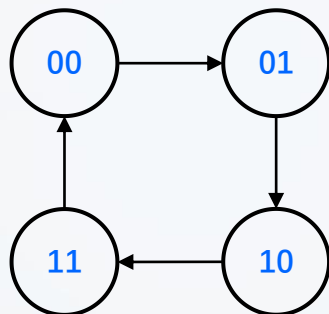


3.2 可靠性编码

3.格雷(Gray)码

十进制数	二进制数	典型格雷码
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

格雷码任意两个相邻数仅有一位不同!

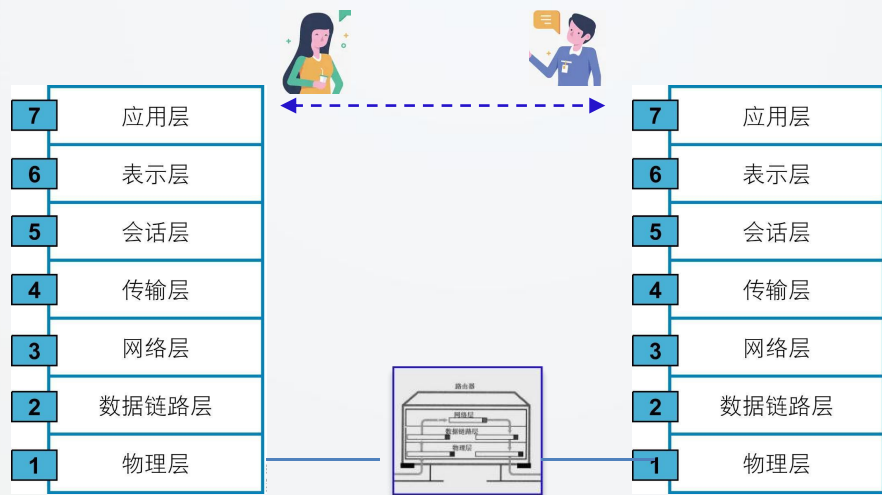


哪种编码的模4计数器更可靠?



1. 为什么要引入校验码

- ◆ 信息在被处理、传输、存储的过程中，受外界环境干扰或器件本身故障影响，可能发生错误；
- ◆ 光纤的数据传输出错率能保持低于 10^{-9} ；
- ◆ 透明性原理，若能设计硬件层的检错和纠错机制，可减少软件或应用层检错的代价(系统观)；



3.3 校验码

2. 校验码的基本原理

No pains , no gains!

◆ 增加冗余码（校验位）

有效信息(k 位) 校验信息(r 位)



3.3 校验码

3. 校验能力与编码的关系

1) 码距的概念

- ◆ 码距: 同一编码体系中, 任意两个合法编码之间二进制数不同位数的最小值
- ◆ 0011与0001 的码距为1, 一位错误时无法识别;
- ◆ 0000、0011、0101、0110、1001、1010、1100、1111等编码码距为2。任何一位发生改变, 如0000变成1000就从有效编码变成了无效编码, 容易检测到这种错误。
- ◆ 校验码中增加冗余位的目的是为了扩大码距。

3.3 校验码

3. 校验能力与编码的关系

2)码距与检错与纠错能力之间的关系

码距	检错(e)	纠错(t)
1	0	0
2	1	0
3	2	或 1
4	2	加 1
5	2	加 2
6	3	加 2
7	3	加 3

◆码距 $\geq e+1$:

可检测e个错误

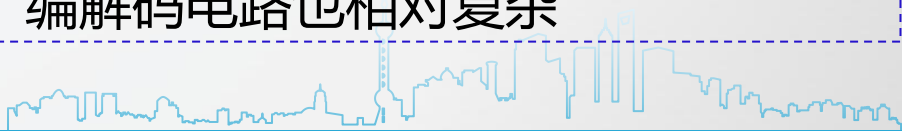
◆码距 $\geq 2t+1$:

可纠正t个错误

◆码距 $\geq e+t+1$:

可纠正t个错误，同时检测e个错误($e>t$)

码距越大，抗干扰能力越强，纠错能力越强，但数据冗余越大，编码效率低，编解码电路也相对复杂



3.3 校验码

4. 奇偶校验

1) 增加冗余校验1位

有效信息(k位) 校验信息(1位)

2) 编码：根据有效信息计算校验信息位，使校验码（数据 + 1位校验信息）中1的个数满足奇/偶数的要求

$0001 \rightarrow 0001\textcolor{blue}{1}$ (偶校验)

$0001 \rightarrow 0001\textcolor{blue}{0}$ (奇校验)

奇校验: $P = \overline{D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_5 \oplus D_6 \oplus D_7 \oplus D_8}$

偶校验: $P = D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_5 \oplus D_6 \oplus D_7 \oplus D_8$

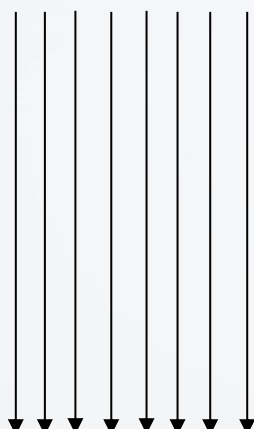
3.3 校验码

3) 检错方法

奇校验检错码: $G = P \oplus X_1 \oplus X_2 \oplus X_3 \oplus X_4 \oplus X_5 \oplus X_6 \oplus X_7$

G=0表示数据正常, 否则表示出错

11000010



11000010

正确传输

11000010



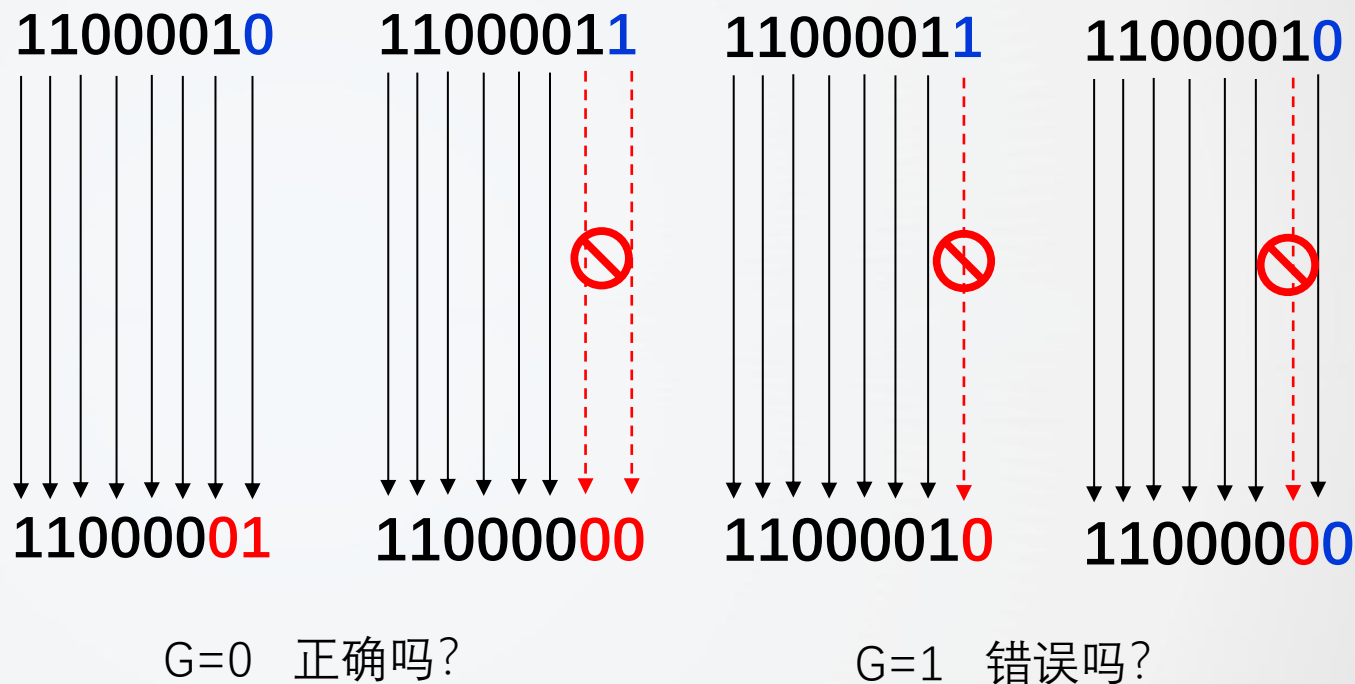
11000011

正常检错

3.3 校验码

4) 奇偶校验的特点

- ◆ 编码与检错简单
- ◆ 编码效率高
- ◆ 不能检测偶数位错误，即无错结论不可信。
- ◆ 不能定位错误，不能纠错，是一种错误检测码。



3.3 校验码

5) 改进的奇偶校验

双向奇偶校验
(Row and Column Parity)

方块校验
垂直水平校验

0110100	1
1011010	0
0010110	1
1110101	1
1001011	0
1000110	1

0110100	1
1011010	0
0010111	1
1110101	1
1001011	0
1000110	1
(可纠错)	
0110100	1
1011010	0
0110110	1
1110101	1
1001001	0
1000110	1
(可检错)	

- ◆可纠正1位错
- ◆可检测出某行(列)上的奇数位错;
- ◆可检出一部分偶数位错
- ◆不能检测出错位分布在矩形4个顶点上的错误

0110100	1
1011010	0
0110100	1
1110101	1
1001011	0
1000110	1
(可检错)	

1110101	1
1011010	0
0010110	1
1110101	1
1001011	0
0000111	1
(不能检错)	

3.3 校验码

6) 奇偶校验应用场景

◆ 哪些场合应用奇偶校验?



哪根内存条最有可能具有奇偶校验功能?

◆ 工程上的应用

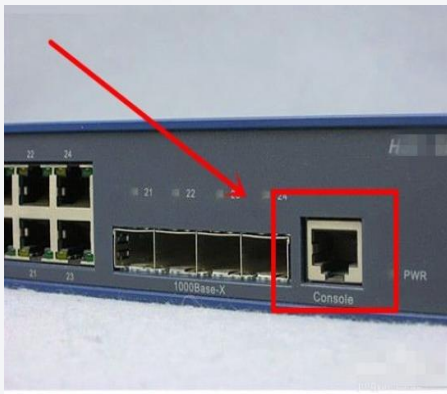
<http://www.eepw.com.cn/article/280413.htm> (关于串口奇偶校验配置的经验)

一般在同步传输方式中常采用奇校验，异步传输方式中常采用偶校验

0000000	00000000	偶校验
	00000001	奇校验

3.3 校验码

6) 奇偶校验应用场景



3.3 校验码

5. CRC校验 (Cyclic Redundancy Check)

1) 增加冗余码 (校验位)

有效信息(k位)

校验信息(r位)

$$N=k+r \leq 2^r - 1$$

2) 生成多项式G(x)

- ◆ 计算校验信息所需要的除数，位数为 $r + 1$ ；
- ◆ 最高位和最低位均为1；
- ◆ 任意1位发生错误都应使余数不为0；
- ◆ 不同位发生错误余数不同；
- ◆ 余数左移一位继续使用生成多项式作模2除，应使余数循环。

3.3 校验码

5. CRC校验 (Cyclic Redundancy Check)

3) 模2运算法则

◆加法/减法：按位加不进位、减不借位，即按位 $\oplus$ 运算

$$0 \pm 0 = 0, \quad 0 \pm 1 = 1, \quad 1 \pm 0 = 1, \quad 1 \pm 1 = 0$$

$$1101 \pm 1110 = 0011 \quad 1101 \oplus 1110 = 0011$$

◆除法：按模2减，求部分余数，不借位



3.3 校验码

5. CRC校验 (Cyclic Redundancy Check)

3) 模2运算法则

◆上商原则

- ①余数首位为1时，商为1，按模2减法运算减除数；
- ②余数首位为0时，商为0，减0；
- ③当余数位数小于除数位数时，该余数即为最后余数。

$$\begin{array}{r}
 1110 \leftarrow \text{商} \\
 \hline
 1011 \overline{) 1100000} \\
 \underline{1011} \\
 1110 \\
 \underline{1011} \\
 1010 \\
 \underline{1011} \\
 0010 \\
 \underline{0000} \\
 010 \leftarrow \text{余数}
 \end{array}$$

3.3 校验码

5. CRC校验 (Cyclic Redundancy Check)

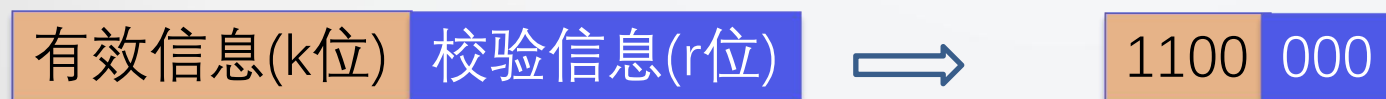
4) CRC 编码方法

(1)根据待校验信息的长度 k , 按照 $k+r \leq 2^r - 1$ 确定校验位 r 的位数

如对4位信息 1100 进行CRC编码, 根据 $4+r \leq 2^r - 1$, 得 $r_{\min} = 3$

(2)根据 r 和生成多项式 G 的关系, 选择位数为 $r + 1$ 的生成多项式 $G(X) = 1011$

(3)对原始信息进行下列变化



即: 将待校验的二进制信息 K 逻辑左移 r 位, 得到 K'

3.3 校验码

(4)对' 按模2运算法则除G(x)，求CRC编码中的r位校验信息

$$\begin{array}{r} 1110 \\ 1011 \overline{) 1100000} \\ \underline{1011} \\ 1110 \\ \underline{1011} \\ 1010 \\ \underline{1011} \\ 0010 \\ \underline{0000} \\ 010 \end{array}$$

(5)用得到的余数替换' 的最后r位即可得到对应的CRC编码



即为1100 的CRC 编码

3.3 校验码

5) CRC检错

接收方利用 $G(X)$ 对收到的编码多项式做模2除运算

$$\begin{array}{r}
 1011 \overline{) 1110010} \\
 \underline{1011} \\
 1110 \\
 \underline{1011} \\
 1011 \\
 \underline{1011} \\
 000
 \end{array}$$

余数为0说明传输没有错误

3.3 校验码

5) CRC检错

当有位出错时，接收方利用G(X)对收到的编码多项式做模2除运算的结果

$$\begin{array}{r} 1110 \\ 1011 \overline{) 1100\color{red}1}10 \end{array}$$

$$\begin{array}{r} 1111 \\ 1011 \overline{) 110\color{red}1}010 \\ \underline{1011} \\ 1100 \\ \underline{1011} \\ 1111 \\ \underline{1011} \\ 1000 \\ \underline{1011} \\ 011 \end{array}$$

$$\begin{array}{r} 1111 \\ 1011 \overline{) 110\color{red}1\color{red}1}10 \\ \underline{1011} \\ 110\color{red}1 \\ \underline{1011} \\ 1101 \\ \underline{1011} \\ 1100 \\ \underline{1011} \\ 111 \end{array}$$

余数不为0说明传输有错

3.3 校验码

5) CRC检错

(7,4) 编码不同数位出错对应的余数

A ₇ ~A ₁	余数	出错位
1100010	000	无
1100011	001	1
1100000	010	2
1100110	100	3
1101010	011	4
1110010	110	5
1000010	111	6
0100010	101	7

$G(X) = 1011$

A ₇ ~A ₁	余数	出错位
1100101	000	无
1100100	001	1
1100111	010	2
1100001	100	3
1101101	101	4
1110101	111	5
1000101	011	6
0100101	110	7

$G(X) = 1101$

- ◆ 不同位出错时，CRC校验得到的不为0余数不同；
- ◆ 同一位出错情况下，CRC校验得到的不为0余数与G有关；
- ◆ 能根据不为0余数唯一确定出错位吗？



3.3 校验码

5) CRC检错

A ₇ ~A ₁	余数	出错位
1100010	000	无
1100011	001	1
1100000	010	2
1100110	100	3
1101010	011	4
1110010	110	5
1000010	111	6
0100010	101	7

1011

1111

110110

1011

1101

1011

1100

1011

111

- (1)能回答上述问题吗?
- (2)还能得到什么结论吗?
- ◆ CRC能检出2位错

◆ CRC能检出3位及以上错误吗?



3.3 校验码

5) CRC检错

(7,4) 编码不同数位出错对应的余数

A ₇ ~A ₁	余数	出错位
1100101	000	无
1100100	001	1
1100111	010	2
1100001	100	3
1101101	101	4
1110101	111	5
1000101	011	6
0100101	110	7

$G(X) = 1101$

◆ 1100 101 -----> 1101 0 0 0
↓
校验结果余数为0

◆ 1100 101 -----> 1101 0 1 0
↓
校验结果余数为010

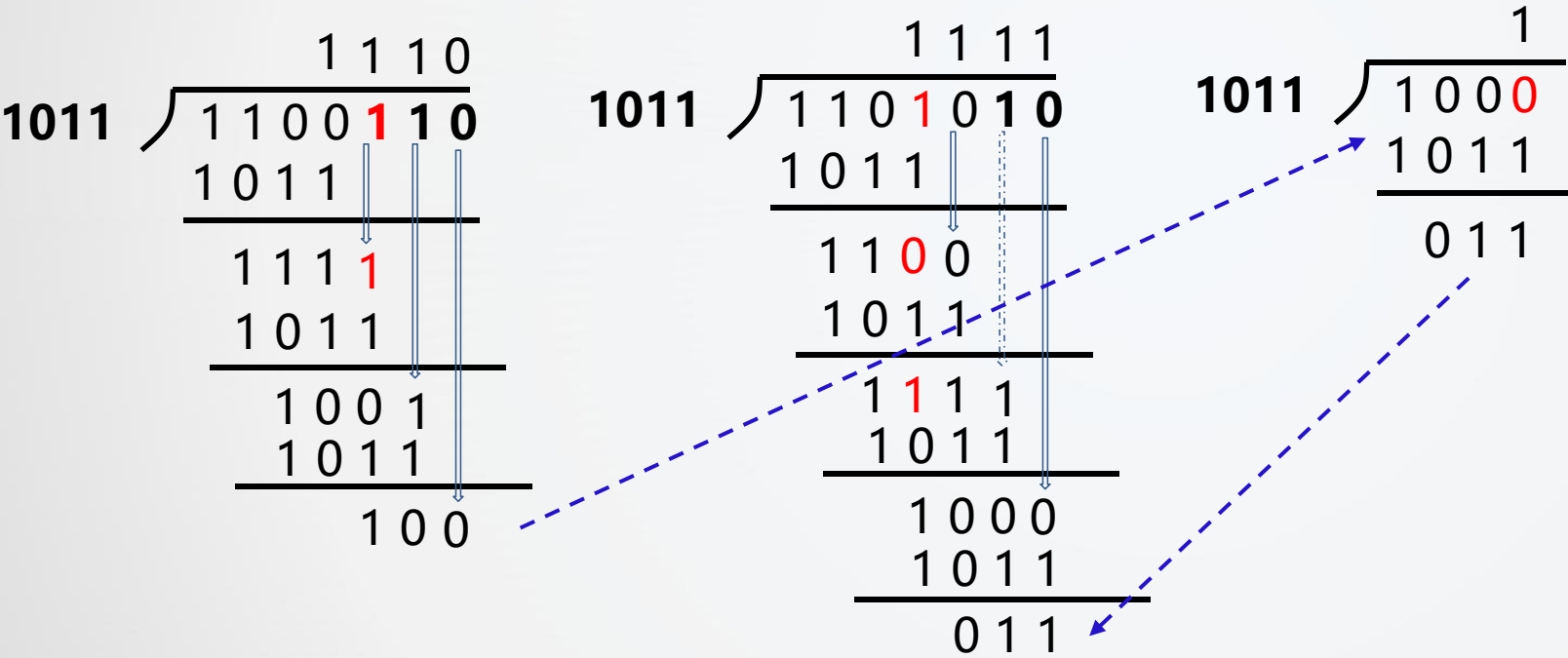
CRC不能检出所有3位出错的情况! (r=4)

在不能确保只有1位错时，无错结论不可信!

3.3 校验码

5) CRC检错

一位出错情况下余数的循环特性

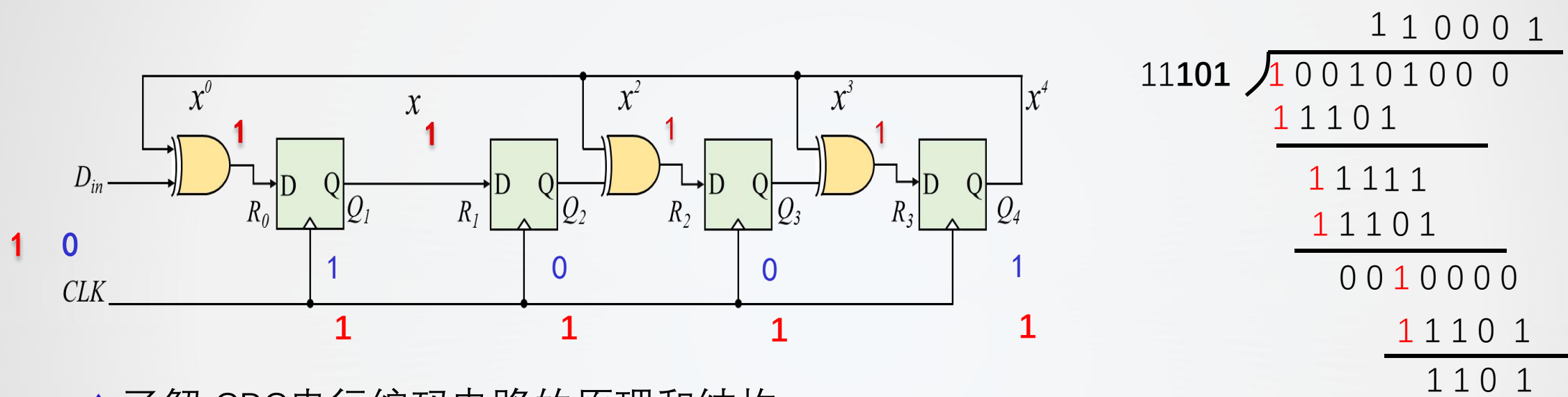


A ₇ ~A ₁	余数	出错的位
1100010	000	无
1100011	001	1
1100000	010	2
1100110	100	3
1101010	011	4
1110010	110	5
1000010	111	6
0100010	101	7

$G(X)=1011$

3.3 校验码

7) CRC编码电路(串行)



- ◆ 了解 CRC串行编码电路的原理和结构
- ◆ 有异或门的位置是生成多项式1项对应的位置, $G(X)=X^4+X^3+X^2+1$ 11101
- ◆ 触发器初始状态为0
- ◆ 最后余数从高到低分别为 $Q_4 \sim Q_1$

3.3 校验码

7) CRC编码电路(并行)

◆ 两数的余数异或等于两数异或后的余数

$$(M(x) \% G(x)) \oplus (N(x) \% G(x)) = (M(x) \oplus N(x)) \% G(x)$$

◆ (7,4) CRC编码, 生成多项式 $G(x)=1011$, 原始数据 $M(x)=1101$, 求CRC编码

1101 000 $\%G(x)$	----->	001
= (1000 000 $\%G(x)$)	----->	101
$\oplus$ (0100 000 $\%G(x)$)	----->	111
$\oplus$ (0001 000 $\%G(x)$)	----->	011

(Note: In the original image, the final result 011 is reached by XORing 111 with 101. A red dashed arrow points from the 101 result to the 011 result.)

◆ 先算(查) 2^6 、 2^5 、 2^4 、 2^3 等常量位的余数, 再将其组合求解任意编码的余数



3.3 校验码

8) 软件实现方式

/******该文件使用查表法计算CCITT 标准的CRC-16检验码，并附测试代码******/

```
#include
```

```
#define CRC_INIT 0xffff //CCITT初始CRC为全1
```

```
#define GOOD_CRC 0xf0b8 //校验时计算出的固定结果值
```

/****下表是常用ccitt 16,生成式OX1021反转成OX8408后的查询表格*****/ (注意，低位先传送时，生成多项式应反转(低位与高位互换))。

```
unsigned short crc16_ccitt_table[256] =
```

```
{ 0x0000, 0x1189, 0x2312, 0x329b, 0x4624, 0x57ad, 0x6536, 0x74bf, 0x8c48, 0x9dc1, 0xaf5a, 0xbed3, 0xca6c, 0xdbe5, 0xe97e, 0xf8f7,
0x1081, 0x0108, 0x3393, 0x221a, 0x56a5, 0x472c, 0x75b7, 0x643e, 0x9cc9, 0x8d40, 0xbfdb, 0xae52, 0xdaed, 0xcb64, 0xf9ff, 0xe876, 0x2102,
0x308b, 0x0210, 0x1399, 0x6726, 0x76af, 0x4434, 0x55bd, 0xad4a, 0xbcc3, 0x8e58, 0x9fd1, 0xeb6e, 0xfae7, 0xc87c, 0xd9f5, 0x3183, 0x200a,
0x1291, 0x0318, 0x77a7, 0x662e, 0x54b5, 0x453c, 0xbdcb, 0xac42, 0x9ed9, 0x8f50, 0xfbef, 0xea66, 0xd8fd, 0xc974, 0x4204, 0x538d, 0x6116,
0x709f, 0x0420, 0x15a9, 0x2732, 0x36bb, 0xce4c, 0xdfc5, 0xed5e, 0xfcd7, 0x8868, 0x99e1, 0xab7a, 0xbaf3, 0x5285, 0x430c, 0x7197, 0x601e,
0x14a1, 0x0528, 0x37b3, 0x263a, 0xdec d, 0xcf44, 0xfddf, 0xec56, 0x98e9, 0x8960, 0xbbfb, 0xaa72, 0x6306, 0x728f, 0x4014, 0x519d, 0x2522,
0x34ab, 0x0630, 0x17b9, 0xef4e, 0xfec7, 0xcc5c, 0xdd d5, 0xa96a, 0xb8e3, 0x8a78, 0x9bf1, 0x7387, 0x620e, 0x5095, 0x411c, 0x35a3, 0x242a,
0x16b1, 0x0738, 0xffcf, 0xee46, 0xdcdd, 0xcd54, 0xb9eb, 0xa862, 0x9af9, 0x8b70, 0x8408, 0x9581, 0xa71a, 0xb693, 0xc22c, 0xd3a5, 0xe13e,
0xf0b7, 0x0840, 0x19c9, 0x2b52, 0x3adb, 0x4e64, 0x5fed, 0x6d76, 0x7cff, 0x9489, 0x8500, 0xb79b, 0xa612, 0xd2ad, 0xc324, 0xf1bf,
0xe036, 0x18c1, 0x0948, 0x3bd3, 0x2a5a, 0x5ee5, 0x4f6c, 0x7df7, 0x6c7e, 0xa50a, 0xb483, 0x8618, 0x9791, 0xe32e, 0xf2a7, 0xc03c,
0xd1b5, 0x2942, 0x38cb, 0x0a50, 0x1bd9, 0x6f66, 0x7eef, 0x4c74, 0x5dfd, 0xb58b, 0xa402, 0x9699, 0x8710, 0xf3af, 0xe226, 0xd0bd,
0xc134, 0x39c3, 0x284a, 0x1ad1, 0x0b58, 0x7fe7, 0x6e6e, 0x5cf5, 0x4d7c, 0xc60c, 0xd785, 0xe51e, 0xf497, 0x8028, 0x91a1, 0xa33a,
0xb2b3, 0x4a44, 0x5bcd, 0x6956, 0x78df, 0x0c60, 0x1de9, 0x2f72, 0x3efb, 0xd68d, 0xc704, 0xf59f, 0xe416, 0x90a9, 0x8120, 0xb3bb,
0xa232, 0x5ac5, 0x4b4c, 0x79d7, 0x685e, 0x1ce1, 0x0d68, 0x3ff3, 0x2e7a, 0xe70e, 0xf687, 0xc41c, 0xd595, 0xa12a, 0xb0a3, 0x8238,
0x93b1, 0x6b46, 0x7acf, 0x4854, 0x59dd, 0x2d62, 0x3ceb, 0x0e70, 0x1ff9, 0xf78f, 0xe606, 0xd49d, 0xc514, 0xb1ab, 0xa022, 0x92b9,
0x8330, 0x7bc7, 0x6a4e, 0x58d5, 0x495c, 0x3de3, 0x2c6a, 0x1ef1, 0x0f78 };
```

```
unsigned short do_crc(unsigned short reg_init, unsigned char *message, unsigned int len)
```

```
{ unsigned short crc_reg = reg_init;
```

```
while (len--)
```

```
    crc_reg = (crc_reg >> 8) ^ crc16_ccitt_table[(crc_reg ^ *message++) & 0xff];
```

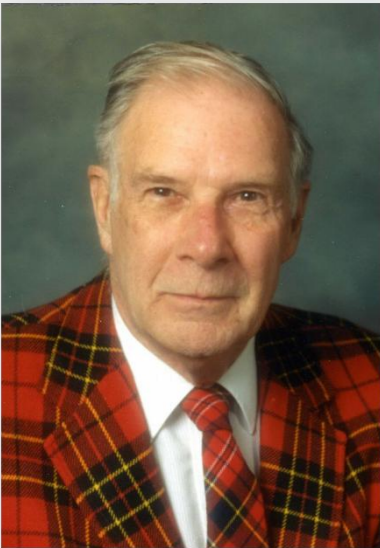
```
return crc_reg; }
```


3.3 校验码

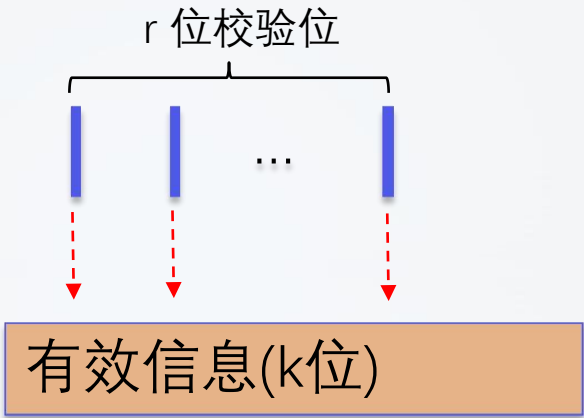
6. 海明校验 (Hamming Codes, 汉明码)

1)海明校验基本概念

◆ 分组奇偶校验 (继承了简单性)



Richard Wesley Hamming 1950



$N = k + r \leq 2^r - 1$

◆

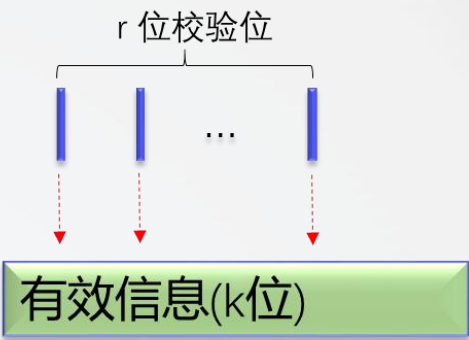
k	1	2 ~ 4	5 ~ 11	12 ~ 26	27 ~ 57	58 ~ 120	...
r	2	3	4	5	6	7	...



3.3 校验码

6. 海明校验 (Hamming Codes, 汉明码)

2) 编码规则



Hi	1	2	3	4	5	6	7	8	9	10	11
	1 /P1	2 /P2	1+2 /b1	4 /P3	1+4 /b2	2,4 /b3	1+2+4 /b4	8 /P4	1+8 /b5	2+8 /b6	1+2+8 /b7

- ◆ k+r位海明码自左至右依次编为第1, 2, 3, ……., k+r位;
- ◆ r 位校验位记为 P_i ($i=1, 2, \dots, r$)分别位于海明编码的第 2^{i-1} 位, 即 H_1 、 H_2 、 H_4 、 H_8 、 H_{16} ,...
- ◆ H_j 位的数据被编号小于j的若干个海明位号之和等于j的校验位所校验;



3.3 校验码

6. 海明校验

2) 编码规则

Hi	1	2	3	4	5	6	7	8	9	10	11
	1 /P1	2 /P2	1+2 /b1	4 /P3	1+4 /b2	2,4 /b3	1+2+4 /b4	8 /P4	1+8 /b5	2+8 /b6	1+2+8 /b7

$P1=b1\oplus b2\oplus b4\oplus b5\oplus b7$

$P2=b1\oplus b3\oplus b4\oplus b6\oplus b7$

$P3=b2\oplus b3\oplus b4$

$P4=b5\oplus b6\oplus b7$



3.3 校验码

6. 海明校验

3) 检错

Hi	1	2	3	4	5	6	7	8	9	10	11
	1 /P1	2 /P2	1+2 /b1	4 /P3	1+4 /b2	2,4 /b3	1+2+4 /b4	8 /P4	1+8 /b5	2+8 /b6	1+2+8 /b7

设置指错字 $G_4G_3G_2G_1$

$$G_4 = P4 \oplus b5 \oplus b6 \oplus b7$$

$$G_3 = P3 \oplus b2 \oplus b3 \oplus b4$$

$$G_2 = P2 \oplus b1 \oplus b3 \oplus b4 \oplus b6 \oplus b7 \quad G_1 = P1 \oplus b1 \oplus b2 \oplus b4 \oplus b5 \oplus b7$$

$G_4G_3G_2G_1$ 为0则表明无错误， 反之给出差错位的海明码位号。



3.3 校验码

6. 海明校验

4)海明校验编码与检错举例

Hi	1	2	3	4	5	6	7	8	9	10	11
	1 /P1	2 /P2	1+2 /b1	4 /P3	1+4 /b2	2,4 /b3	1+2+4 /b4	8 /P4	1+8 /b5	2+8 /b6	1+2+8 /b7

设被传送的信息 $b_1b_2b_3b_4b_5b_6b_7 = 1\ 0\ 1\ 1\ 0\ 0\ 0$ ，采用偶校验；

$$\text{则: } P1=b1\oplus b2\oplus b4\oplus b5\oplus b7 = 1\oplus 0\oplus 1\oplus 0\oplus 0 = 0$$

$$P2=b1\oplus b3\oplus b4\oplus b6\oplus b7 = 1\oplus 1\oplus 1\oplus 0\oplus 0=1$$

$$P3=b2\oplus b3\oplus b4 = 0\oplus 1\oplus 1 = 0$$

$$P4=b5\oplus b6\oplus b7 = 0\oplus 0\oplus 0 = 0$$

得到的海明编码为 $H = 0\ 1\ 1\ 0\ 0\ 1\ 1\ 0\ 0\ 0\ 0$



3.3 校验码

6. 海明校验

4)海明校验编码与检错举例

Hi	1	2	3	4	5	6	7	8	9	10	11
	1 /P1	2 /P2	1+2 /b1	4 /P3	1+4 /b2	2,4 /b3	1+2+4 /b4	8 /P4	1+8 /b5	2+8 /b6	1+2+8 /b7

当传输无错时，即H = 0 1 1 0 0 1 1 0 0 0 0

则： $G_4 = P_4 \oplus b_5 \oplus b_6 \oplus b_7 = 0 \oplus 0 \oplus 0 \oplus 0 = 0$

$G_3 = P_3 \oplus b_2 \oplus b_3 \oplus b_4 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$

$G_2 = P_2 \oplus b_1 \oplus b_3 \oplus b_4 \oplus b_6 \oplus b_7 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 \oplus 0 = 0$

$G_1 = P_1 \oplus b_1 \oplus b_2 \oplus b_4 \oplus b_5 \oplus b_7 = 0 \oplus 1 \oplus 0 \oplus 1 \oplus 0 \oplus 0 = 0$

$G_4 G_3 G_2 G_1 = 0 0 0 0$ ， 表明无错!



3.3 校验码

6. 海明校验

4)海明校验编码与检错举例

Hi	1	2	3	4	5	6	7	8	9	10	11
	1 /P1	2 /P2	1+2 /b1	4 /P3	1+4 /b2	2,4 /b3	1+2+4 /b4	8 /P4	1+8 /b5	2+8 /b6	1+2+8 /b7

当传输出错时，即H = 0 1 1 0 0 1 1 0 0 0 1

则： $G_4 = P_4 \oplus b_5 \oplus b_6 \oplus b_7 = 0 \oplus 0 \oplus 0 \oplus 1 = 1$

$G_3 = P_3 \oplus b_2 \oplus b_3 \oplus b_4 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$

$G_2 = P_2 \oplus b_1 \oplus b_3 \oplus b_4 \oplus b_6 \oplus b_7 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 \oplus 1 = 1$

$G_1 = P_1 \oplus b_1 \oplus b_2 \oplus b_4 \oplus b_5 \oplus b_7 = 0 \oplus 1 \oplus 0 \oplus 1 \oplus 0 \oplus 1 = 1$

$G_4 G_3 G_2 G_1 = 1 0 1 1$ ， 表明H₁₁位出错!



3.3 校验码

6. 海明校验

5)海明校验特点分析

Hi	1	2	3	4	5	6	7	8	9	10	11
	1 /P1	2 /P2	1+2 /b1	4 /P3	1+4 /b2	2,4 /b3	1+2+4 /b4	8 /P4	1+8 /b5	2+8 /b6	1+2+8 /b7

(1) 指错字 $G_4G_3G_2G_1= 0000$ 一定无错吗?

当传输无错时, 即 $H = 0\ 1\ 1\ 0\ 0\ 1\ 1\ 0\ 0\ 0\ 0$

$$G_4 = P_4 \oplus b_5 \oplus b_6 \oplus b_7 = 0 \oplus 0 \oplus 0 \oplus 0 = 0$$

$$G_3 = P_3 \oplus b_2 \oplus b_3 \oplus b_4 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$$

$$G_2 = P_2 \oplus b_1 \oplus b_3 \oplus b_4 \oplus b_6 \oplus b_7 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 \oplus 0 = 0$$

$$G_1 = P_1 \oplus b_1 \oplus b_2 \oplus b_4 \oplus b_5 \oplus b_7 = 0 \oplus 1 \oplus 0 \oplus 1 \oplus 0 \oplus 0 = 0$$

能发现什么规律吗?



3.3 校验码

6. 海明校验

2) 指错字能区别一位错和多位错吗？

$$G_4 = P_4 \oplus b_5 \oplus b_6 \oplus b_7 = 0 \oplus 0 \oplus 0 \oplus 0 = 0$$

$$G_3 = P_3 \oplus b_2 \oplus b_3 \oplus b_4 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$$

$$G_2 = P_2 \oplus b_1 \oplus b_3 \oplus b_4 \oplus b_6 \oplus b_7 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 \oplus 0 = 0$$

$$G_1 = P_1 \oplus b_1 \oplus b_2 \oplus b_4 \oplus b_5 \oplus b_7 = 0 \oplus 1 \oplus 0 \oplus 1 \oplus 0 \oplus 0 = 0$$

$$b_1、b_2 \text{ 出错时: } G_4 G_3 G_2 G_1 = 0110 \quad b_5、b_6、b_7 \text{ 出错时: } G_4 G_3 G_2 G_1 = 1000$$

$$\text{仅 } b_3 \text{ 出错时: } G_4 G_3 G_2 G_1 = 0110 \quad \text{仅 } P_4 \text{ 出错时: } G_4 G_3 G_2 G_1 = 1000$$



3.3 校验码

6. 海明校验

3)指错字能区别多种不同多位同时出错的情况吗？

$$G_4 = P_4 \oplus b_5 \oplus b_6 \oplus b_7 = 0 \oplus 0 \oplus 0 \oplus 0 = 0$$

$$G_3 = P_3 \oplus b_2 \oplus b_3 \oplus b_4 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$$

$$G_2 = P_2 \oplus b_1 \oplus b_3 \oplus b_4 \oplus b_6 \oplus b_7 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 \oplus 0 = 0$$

$$G_1 = P_1 \oplus b_1 \oplus b_2 \oplus b_4 \oplus b_5 \oplus b_7 = 0 \oplus 1 \oplus 0 \oplus 1 \oplus 0 \oplus 0 = 0$$

b_1 、 b_2 出错时 与 P_2 、 P_3 同时出错时候得到的指错字相同 (0110)

总结论：在能确定只有一位出错时，出错结论可靠且能定位错误；否则，出错结论可靠但不能定位，**无错结论不可靠。**



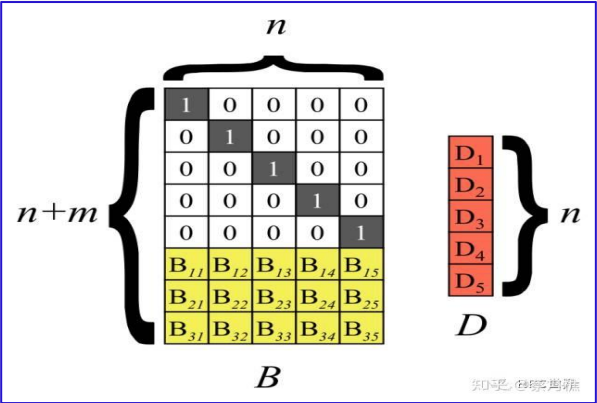
3.3 校验码

7. 校验码应用

◆ RAID(磁盘阵列)



◆ 纠删码 (erasure coding, EC, 云时代RAID的延展)



□1	纠删码集群内存空间回收方案研究	廖宝忠	华中科技大学	2021-05-01	硕士
□2	感知风险与热度的纠删码内存恢复方案研究	冯慧君	华中科技大学	2021-05-01	硕士
□3	面向分布式存储系统数据快速修复的纠删码技术研究	叶柳青	华中科技大学	2020-09-01	博士
□4	基于纠删码的云存储系统扩展与修复性能的研究	张晓阳	华中科技大学	2020-08-01	博士
□5	面向数据去重系统的分层纠删码算法设计与实现	刘振池	华中科技大学	2020-06-01	硕士
□6	数据去重系统中高可扩展性纠删码算法的设计与优化	周嘉伟	华中科技大学	2020-06-01	硕士
□7	基于纠删码的SSD自适应容错方案及其性能优化	王先鹏	华中科技大学	2020-06-01	硕士
□8	基于纠删码的分布式固态硬盘键值存储系统优化设计	薛慷	华中科技大学	2019-05-01	硕士
□9	纠删码系统中基于异或网络计算的数据修复及更新优化研究	唐英杰	华中科技大学	2019-05-01	硕士
□10	纠删码存储系统中基于网络计算的高效故障重建方法	唐英杰; 王芳; 谢燕文	计算机研究与发展	2019-04-15	期刊
□11	纠删码内存集群更新方案研究	夏杰	华中科技大学	2018-05-01	硕士
□12	基于SSD的分布式存储系统纠删码方案设计和优化	彭育鹏	华中科技大学	2018-05-01	硕士
□13	基于纠删码的HDFS的优化方法的设计与实现	游艳萍	华中科技大学	2017-12-01	硕士
□14	分布式存储系统中数据快速修复的纠删码	柳青	华中科技大学	2017-05-01	博士
□15	纠删码存储集群的在线扩容方案研究	黄文胜	华中科技大学	2017-05-01	硕士
□16	纠删码存储集群的数据安全性研究	蔡颖	华中科技大学	2017-05-01	硕士

(2023-3-8:28篇)
(2025-3-4:40篇)